

2교시 · RAG 1 — AI에게 자료를 전달하는 구조

왜 RAG인가

1교시 채팅에서 AI는 날씨를 원래 알던 게 아니라, **Open-Meteo 값을 질문과 함께 받아서** 읽고 답했다. 파일 1개 첨부는 되지만, 참고 자료가 100개·1000개면? 매번 전부 첨부할 수 없다 — 이 질문에서 출발한 구조가 RAG.

RAG = Retrieval Augmented Generation (검색 증강 생성)

- **검색(Retrieval)** — 질문과 관련된 자료를 저장소에서 찾음
- **증강(Augmented)** — 찾은 자료를 질문과 함께 AI에게 넘김
- **생성(Generation)** — AI가 넘겨받은 자료를 근거로 답 생성

핵심: 문서 전체를 AI에게 넘기지 않는다. 질문에 필요한 조각만 골라 넘긴다.

naive RAG 구성 — "저장할 때 셋, 물어볼 때 둘"

저장 단계 (문서를 처음 넣을 때 한 번만)

1. **파싱(Parsing)** — PDF에서 글자 추출
2. **청킹(Chunking)** — 다루기 좋은 길이로 자름 (조각 = 청크)
3. **임베딩(Embedding)과 적재** — 청크를 숫자 배열로 변환해 저장소에 저장

질문 단계 (질문마다 매번)

4. **검색** — 질문도 숫자 배열로 변환 → 저장된 청크와 거리 계산 → 가까운 것 추출
5. **생성** — 추출한 청크 + 질문 → 답 생성

임베딩이 하는 일

컴퓨터는 "두 문장이 비슷한 뜻인가"를 직접 판단 못함 → 문장을 숫자 배열로 변환해 거리로 판단. 이 변환 모델이 **임베딩 모델**.

```
"회의실 예약은 사내 포털에서 합니다" → [0.12, -0.83, 0.44, ...]
"미팅룸은 인트라넷으로 잡으세요" → [0.11, -0.79, 0.41, ...] (뜻이 비슷 → 숫자도 가까움)
"주차권은 안내데스크에서 받습니다" → [-0.67, 0.25, -0.90, ...]
```

- 오늘 쓰는 모델: OpenAI `text-embedding-3-small` (1교시 API 키 재사용). 문장 1개 → 숫자 1,536개.
- 질문과 문서는 반드시 같은 모델로 임베딩 — 다른 모델이면 거리 계산 자체가 불가능.

rag-lab 앱 실습

1. `rag-lab.zip` 압축 해제 → `.env.example` 복사해 `.env` 생성, Key 입력
2. `cd rag-lab` → `claude` 실행 → `/init` (코드를 직접 읽지 않고 구조 파악)
 이 앱은 무엇을 하는 프로그램이야? 파싱, 청킹, 임베딩, 검색이 각각 어느 파일에서 일어나는지 알려줘.
 이 앱을 실행하려면 무엇이 필요해? 필요한 패키지가 있으면 설치해 줘.
 (chromadb · pypdf 등 설치)
3. `run.bat` 관리자 권한으로 실행 → 브라우저에 4개 탭 (RAG 단계와 대응)

naive RAG 단계별 관찰

① 파싱 — 문서 3종 업로드

- 반도체 품질 핸드북(도판·표多) / 소프트웨어 응용노트(절 번호 체계) / MSL 패키지 응용노트(사진·그래프多)
- 막대그래프 = 페이지당 추출 글자 수. 붉은 칸 = 글자 거의 없음 → 원문 대조로 무엇이 누락됐는지 확인 (표/그림이 텍스트로 안 들어온 경우가 혼함)

② 청킹 — 3가지 기준 비교

- 고정 길이 — 글자 수로 기계적으로 자름
- 구조 경계 — 절 번호·제목 줄에서 자름
- 문단 — 빈 줄 기준 최소 단위

자르는 길이에 정답 없음: 크게 자르면 맥락 유지되지만 무관한 내용도 딸려옴, 작게 자르면 정확하지만 문장이 자주 끊김. 문서 성격을 보고 사람이 판단.

③ 임베딩과 적재

[적재] 클릭 → 청크가 임베딩 모델을 거쳐 숫자 배열로 저장. 완료 후 청크 번호로 실제 숫자 배열 확인 가능.

④ 의미 검색

질문("1 FIT 는 무엇을 뜻하나요?")을 한국어/영어로 각각 던져 비교. 문서가 전부 영어일 때 한국어 질문이 근거를 잘 찾아오는지 실무의 판단 근거(번역본을 따로 넣어야 하는지 등).

⑤ 생성

[답변 생성] ON → 검색된 청크 + 질문 → 답변. 프롬프트 글자 수(=실제 AI가 읽은 양) vs 저장소 전체 글자 수를 비교 — 답변 품질은 곧 검색 품질.

- 문서에 없는 질문을 일부러 던져 환각 여부 확인
- 그림/표 내용(글자로 안 들어온 정보)에 대해 질문해 근거 대조

사람은 문장으로만 묻지 않는다 — 키워드 검색(BM25)

단어만 입력해보면(`FIT` , `ALPS` , `6.2.1.1` , `Rev 2` , `박리`) 의미 검색이 놓치는 경우가 나타남.

BM25 추가 (Claude Code에게 개발 요청)

이 앱의 검색 화면에 BM25 검색을 추가해줘.
검색 화면에서 의미 검색과 BM25 검색 결과를 좌우로 나란히 볼 수 있게 구현해줘.
답변 생성 선택 후 검색 수행 시, 의미 검색과 BM25 검색의 청크를 사용한 각각의 답변을 보여줘.

⚠ 코드가 바뀌면 앱을 **통째로 재시작**해야 반영됨: 검은 창에서 `Ctrl+C` → `run.bat` 재실행(관리자 권한). 이후 모든 개발 요청에 공통.

BM25 = 흔한 낱말보다 드문 낱말이 겹칠 때 점수를 크게 주는 키워드 매칭 방식.

하이브리드 검색 (의미 + BM25 결합)

의미 검색과 BM25 의 결과를 하나로 합치는 하이브리드 검색을 추가해 줘.
점수를 그대로 더하지 말고 순위로 결합해 줘. 두 점수는 계산법이 달라서 크기 단위가 달라.
검색 화면에 [의미 검색] [BM25] [하이브리드] 세 열을 나란히 보여 줘.
답변 생성 선택 후 검색 수행 시, 각 검색의 청크를 사용한 각각의 답변을 보여줘.

점수를 그대로 더치지 말고 **순위로 결합**하는 이유: 계산법이 달라 점수 크기 단위가 다름.

문서가 늘어나면?

문서 2개(`materials_ch3`) 추가 적재 후 질문:

AEC-Q100 Grade 1 의 HTOL 시험은 몇 시간 동안 하나요?

→ 답변, 근거 5개 청크의 출처, 눈에 걸리는 점을 기록. 여기서 발견한 문제(같은 항목인데 문서 판(Rev)이 달라 값이 다름)가 3교시의 출발점.

정리

- RAG 5단계를 직접 관찰: 파싱·청킹·임베딩·적재·검색·생성
- 자르는 기준, 찾는 방식은 **사람이 정하는 값** — 문서 종류/상태를 보고 결정
- 답의 품질은 **어떤 조각을 꺼냈는가**에 거의 달려 있음

지금 가진 것: `rag-lab` — 파싱·청킹 3종·임베딩·적재·의미검색·답생성 + 직접 만든 BM25·하이브리드 / 저장소: `materials_ch2`(3종) + `materials_ch3`(2종)